

第八周——自定义函数

题目目的

- (一) 掌握R函数的定义方法。
- (二) 掌握R运算符的定义方法。
- (三) 掌握自定义函数与运算符的简单应用。
- (四) 掌握递归函数的定义方法。

题目

题目一：自定义函数。打开脚本文件test0801.R，完成下面操作。

- (1) 创建自定义函数count_even，函数的输入参数为整数型向量，输出为向量中偶数的数目。
- (2) 首先需要对参数进行检查，检查内容包括：
 - 是否缺失参数，若缺失，则终止调用，并给出错误信息“没有输入被检测向量！”
 - 检查数据对象，若不是向量，则终止调用，并给出错误信息“输入数据对象不是向量！”
 - 检查数据类型，若不是数值型数据，则返回0，并给出警告信息“输入数据不是数值型数据！”
- (3) 要求用循环语句实现计算。
- (4) 运行脚本程序后面的测试代码，观察结果是否正确。

```
# 在下面空白定义函数count_even，
# 然后统计向量x中偶数个数

# 运行下面的代码，
# 观察结果是否满足题目要求
x = round(runif(20,1,100),digits = 0)
print(x)
```

```
[1] 50 77 36 24 5 40 86 22 62 75 45 16 94 58 100 76 70 52 31
[20] 94
```

```
y = 20
z = c(FALSE, TRUE, FALSE, TRUE, TRUE)
w = matrix(1:12,nrow = 3)

#count_even(x)
#count_even()
#count_even(y)
#count_even(z)
```

题目二：自定义运算符。打开脚本文件test0802.R，完成下面操作。

- (1) 自定义运算符%xor%，计算0与1的异或操作，其运算规则如下：

Note

1 %xor% 1 = 0, 0 %xor% 0 = 0

1 %xor% 0 = 1, 0 %xor% 1 = 1

(2) 若输入数据不是0或1，则返回NA。

(3) 运行脚本程序后面的测试代码，观察结果是否满足题目要求。

```
# 在下方空白处定义运算符
```

```
# 运行下面的测试程序
```

```
#0%xor%0
```

```
#0%xor%1
```

```
#1%xor%0
```

```
#1%xor%1
```

```
#2%xor%0
```

```
#1%xor%3
```

题目三：自定义函数及其参数的默认值。打开脚本文件test0803.R，完成下面操作。

(1) 编写一个自定义函数matrix_compute，该函数的输入参数为矩阵mat和标识参数flag。根据flag值，对矩阵进行计算，计算规则为：

- 若flag = 1，则计算mat中每行元素的平方和；□若flag = 2，则计算mat中每列元素的平方和；
- 若flag = 3，则计算mat中所有元素的平方和。
- 若flag为其它值，则返回NA

(2) 将标识参数flag的默认值设置为3。

(3) 运行脚本程序后面的测试代码，观察结果是否正确。

```
# 在下方空白处定义函数matrix_compute
```

```
# 运行下面的测试代码，检查运算结果是否正确
```

```
set.seed(1)
```

```
x = floor(runif(12,1,10))
```

```
x = matrix(x,nrow = 3)
```

```
print(x)
```

```
      [,1] [,2] [,3] [,4]  
[1,]    3    9    9    1  
[2,]    4    2    6    2  
[3,]    6    9    6    2
```

```
#matrix_compute(x, flag = 1)
```

```
#matrix_compute(x, flag = 2)
```

```
#matrix_compute(x, flag = 3)
```

```
#matrix_compute(x, flag = 4)
```

```
#matrix_compute(x)
```

题目四：省略号参数的使用。打开脚本文件test0804.R，完成下面操作。

(1) 自定义函数stat_compute，该函数有两个输入参数：

- 第一个参数fun是字符串，其值可设置为‘sum’、‘median’、‘mean’、‘var’，分别表示求和、求中位数、求平均值、求方差；
- 第二个参数为省略号参数。

(2) 该函数根据第一个参数的意义，计算省略号参数所代表的全体元素的和、中位数、平均值或方差。

(3) 运行脚本程序后面的测试代码，验证结果是否正确。

```
# 在下方空白处定义函数stat_compute
```

```
# 运行下面的测试代码，检查运算结果是否正确
x = c(3, 9, 4, 7, 2)
#stat_compute('sum', 3, 9, 4, 7, 2)
sum(x)
```

[1] 25

```
#stat_compute('median', 3, 9, 4, 7, 2)
median(x)
```

[1] 4

```
#stat_compute('mean', 3, 9, 4, 7, 2)
mean(x)
```

[1] 5

```
#stat_compute('var', 3, 9, 4, 7, 2)
var(x)
```

[1] 8.5

题目五：自定义函数的应用1。打开脚本文件test0805.R，完成下面操作。

(1) 定义函数myfuture(*P*,*r*,*y*,*t*)，该函数用于计算银行存款的本利之和，计算公式如下：

$$F = P(1 + \frac{r}{100t})^{ty}$$

其中 F 是本利之和， P 是本金， r 是年利率（比如说是2.7，它表示年利率是2.7%）， t 表示计息周期（ $t = 12$ ，表示每月计息一次）， y 表示存储的年数。

(2) 参数分别为 P 、 r 、 y 、 t ， r 其中的默认值为2.7；若缺失参数，则终止运行，并给出错误信息“未输入存款的年数”；若缺失参数 t ，则赋值为12，并给出警告信息“参数 t 被设置为12”。

(3) 用Excel的FV函数验证结果的正确性。

```
# 在下方空白处定义函数myfuture
```

```
# 运行下面的测试代码
#myfuture(10000,y = 5)
#myfuture(10000,r = 2.4)
#myfuture(10000, r = 2.5, y = 10, t = 12)
# Excel的验证公式如下
# =FV(0.025/12, 120, 0,10000,0)
```

题目六：自定义函数的应用 2。打开脚本文件test0806.R，完成下面操作。

(1) 定义函数myvar，输入参数为向量x，函数的返回值为x的方差，计算公式为：

$$var = \frac{a}{n-1} \sum_{k=1}^n (x_k - \bar{x})^2$$

其中 n 为向量x的长度， x_k 为向量x中的第 k 个元素。

(2) 要求用循环语句实现（不能使用var函数或cor函数）， $\bar{x}$ 可使用mean计算。

(3) 运行脚本程序后面的测试代码，验证结果是否正确。

```
# 在下方空白处定义函数myvar
```

```
# 运行下面的测试代码，检验结果
x = rnorm(20,0,2)
#myvar(x)
var(x)
```

```
[1] 3.576572
```

题目七：自定义函数的应用3。打开脚本文件test0807.R，完成下面操作。

(1) 定义函数bin2dec(sbin)，其功能是将二进制字符串转换成十进制整数，其中输入参数sbin是0和1构成的字符串，返回值为对应的整数，例如，输入为“101011”，输出为43)。

(2) 需要对参数进行检查，若包含非0与非1字符，则停止运行，给出错误信息“输入包含了非0-1字符串”。提示：用正则表达式和grepl函数。

(3) 转换规则为提取字符串中每个字符，然后表示为：

$$x_n \times 2^{n-1} + x_{n-1} \times 2^{n-2} + \dots + x_2 \times 2^1 + x_1 \times 2^0$$

其中 x_k 为字符串中的第 k 个字符（从右向左数），例如，“101011”的十进制整数为：

$$1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

(4) 算法过程分析

- 假设初始累加器 $s = 0$ ，那么：

$$1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = (((((s \times 2 + 1) \times 2 + 0) \times 2 + 1) \times 2 + 0) \times 2 + 1) \times 2 + 1$$

- 每次循环 s 先乘以2，然后加上新提取的字符（从左向右提取）。

💡 Tip

提示：用substr或substring函数提取字符。

```
# 请下方空白定义函数bin2dec
```

```
#运行下面代码，检验程序结果
#bin2dec("1021") # 错误
#bin2dec("1001") # 9
#bin2dec("101011") # 43
```

题目八：自定义函数的应用4。打开脚本文件test0808.R，完成下面操作。

(1) 定义函数dec2bin(x)，其中x是正整数，函数的返回值为x的二进制数字字符串（0-1字符串），例如，dec2bin(3)的结果是“10”，dec2bin(10)的结果是“1010”。

(2) 转换方法是“求余数，自下向上”。例如，13的转换过程为：

- 13除以2的余数是1，商是6，得到第一个余数是1；
- 6（上一步的商）除以2的余数是0，商是3，得到第二个余数是0；
- 3（上一步的商）除以2的余数是1，商是1，得到第三个余数是1；
- 1（上一步的商）除以2的余数是1，商是0，得到第四个余数是1；
- 上一步的商是0，则停止，自下向上取余数，得到输出“1101”

💡 Tip

提示：用`ceiling(log2(x+1))`计算二进制位数，想想为什么？

```
# 在下面空白处定义函数dec2bin
```

```
# 运行下面代码，验证结果
```

```
#dec2bin(14) # 1110
```

```
#dec2bin(123) # 1111011
```

```
#dec2bin(2203) # 100010011011
```

题目九：简单递归函数。打开脚本文件test0809.R，完成下面操作。

(1) 定义函数mysum(n)，其中n是大于1整数，函数返回

$$1 + 2 + \dots + n$$

的值。

(2) 要求用递归完成（不用循环语句）。

(3) 用mysum(100)测试，其结果是否为5050。

```
# 在下面空白处定义函数mysum
```

```
# 测试
```

```
#mysum(100)
```

题目十：递归函数的应用1。打开脚本文件test0810.R，完成下面操作。

定义函数myrecur(x,n)，该函数返回x中第n个数字（从右向左算），其中参数x和n是正整数，例如myrecur(123456, 3)的结果是4，myrecur(123456, 6)的结果是1，myrecur(123456,7)的结果是0。

```
# 在下面空白处定义函数myrecur
```

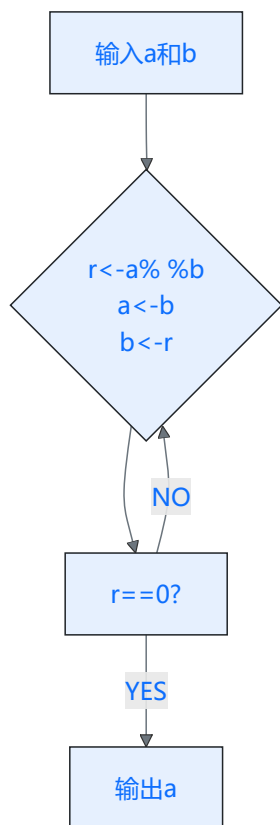
```
# 测试
```

```
#myrecur(3627854,3) #8
```

```
#myrecur(3627854,6) #6
```

题目十一：递归函数的应用2。打开脚本文件test0811.R，完成下面操作。

(1) 定义函数mygcd(a,b)，该函数用欧几里德算法计算两整数a和b的最大公约数，流程图如下图所示。



(2) 与上周的实验题9比较，请说明递归的优势。

答案及解析

题目一：

```
# 在下面空白定义函数count_even,
# 然后统计向量x中偶数个数
count_even <- function(x){
  if (missing(x)){
    stop('没有输入被检测向量! ')
  }
  if (!is.vector(x)){
    stop('输入数据对象不是向量! ')
  }
  if (!is.numeric(x)){
    warning('输入数据不是数值型数据! ')
    return(0)
  }
  count <- 0
  for (i in x){
    if (i%%2==0){
      count <- count + 1
    }
  }
  return(count)
}

# 运行下面的代码,
# 观察结果是否满足题目要求
x = round(runif(20,1,100),digits = 0)
print(x)
```

```
[1] 44 25  8 11 32 52 67 41 91 30 46 34 65 27 48 77  9 88 35 84
```

```
y = 20
z = c(FALSE, TRUE, FALSE, TRUE, TRUE)
w = matrix(1:12,nrow = 3)

count_even(x)
```

```
[1] 10
```

```
#count_even()
count_even(y)
```

```
[1] 1
```

```
count_even(z)
```

Warning in count_even(z): 输入数据不是数值型数据！

```
[1] 0
```

题目二:

```
# 在下方空白定义运算符
'%xor%' <- function(x,y){
  if (!(x %in% c(0, 1)) || !(y %in% c(0, 1))){
    return(NA)
  }
  if(x == 1 && y == 1){
    return(0)
  } else if (x == 0 && y == 0){
    return(0)
  } else {
    return(1)
  }
}

# 运行下面的测试程序
0%xor%0
```

[1] 0

```
0%xor%1
```

[1] 1

```
1%xor%0
```

[1] 1

```
1%xor%1
```

[1] 0

```
2%xor%0
```

[1] NA

```
1%xor%3
```

[1] NA

题目三:

```
# 在下方空白处定义函数matrix_compute
matrix_compute <- function(x, flag = 3){
  squared_mat <- x^2
  if (flag == 1){
    return(rowSums(squared_mat))
  }else if(flag == 2){
    return(colSums(squared_mat))
  }else if(flag == 3){
    return(sum(squared_mat))
  }else{
    return(NA)
  }
}

# 运行下面的测试代码，检查运算结果是否正确
set.seed(1)
x = floor(runif(12,1,10))
x = matrix(x,nrow = 3)
print(x)
```

```
      [,1] [,2] [,3] [,4]
[1,]    3    9    9    1
[2,]    4    2    6    2
[3,]    6    9    6    2
```

```
matrix_compute(x, flag = 1)
```

```
[1] 172  60 157
```

```
matrix_compute(x, flag = 2)
```

```
[1]  61 166 153   9
```

```
matrix_compute(x, flag = 3)
```

```
[1] 389
```

```
matrix_compute(x, flag = 4)
```

```
[1] NA
```

```
matrix_compute(x)
```

```
[1] 389
```

题目四:

```
# 在下方空白处定义函数stat_compute
stat_compute <- function(fun, ...) {
  allowed_funs <- c("sum", "median", "mean", "var")
  fun <- match.arg(fun, allowed_funs)
  args <- list(...)
  if (length(args) == 0) {
    stop("没有提供数据进行计算！")
  }
  data <- unlist(args)
  result <- switch(fun,
    sum = sum(data),
    median = median(data),
    mean = mean(data),
    var = var(data))

  return(result)
}

# 运行下面的测试代码，检查运算结果是否正确
x = c(3, 9, 4, 7, 2)
stat_compute('sum', 3, 9, 4, 7, 2)
```

[1] 25

```
sum(x)
```

[1] 25

```
stat_compute('median', 3, 9, 4, 7, 2)
```

[1] 4

```
median(x)
```

[1] 4

```
stat_compute('mean', 3, 9, 4, 7, 2)
```

[1] 5

```
mean(x)
```

[1] 5

```
stat_compute('var', 3, 9, 4, 7, 2)
```

[1] 8.5

```
var(x)
```

[1] 8.5

题目五:

```
# 在下方空白处定义函数myfuture
myfuture <- function(P,r=2.7,y,t){
  if (missing(y)){
    stop("未输入存款的年数")
  }
  if (missing(t)){
    t <- 12
    warning("参数被设置为12")
  }
  future_value <- ((1 + (r / 100 / t))^(t * y)) * P
  return(future_value)
}

# 运行下面的测试代码
myfuture(10000,y = 5)
```

Warning in myfuture(10000, y = 5): 参数被设置为12

[1] 11443.63

```
#myfuture(10000,r = 2.4)
myfuture(10000, r = 2.5, y = 10, t = 12)
```

[1] 12836.92

```
# Excel的验证公式如下
# =FV(0.025/12, 120, 0,10000,0)
```

题目六:

```
# 在下方空白处定义函数myvar
myvar <- function(x){
  x_hat <- mean(x)
  total = 0
  for (i in x) {
    total <- total +(i - x_hat)^2
  }
  result <- total / (length(x)-1)
  return(result)
}

# 运行下面的测试代码，检验结果
x = rnorm(20,0,2)
myvar(x)
```

[1] 3.576572

```
var(x)
```

[1] 3.576572

题目七:

```
# 请下方空白定义函数bin2dec
bin2dec <- function(sbin){
  if (grepl('[^01]',sbin)){
    stop('输入包含了非0-1字符串')
  }
  decimal_value <- as.integer(strtoi(sbin, base = 2))
  return(decimal_value)
}
```

```
#运行下面代码，检验程序结果
#bin2dec("1021") # 错误
bin2dec("1001") # 9
```

```
[1] 9
```

```
bin2dec("101011") # 43
```

```
[1] 43
```

题目八:

```
# 在下面空白处定义函数dec2bin
dec2bin <- function(x){
  binary_string <- ''
  while (x>0) {
    remainder <- x%%2
    binary_string <- paste0(remainder,binary_string)
    x <- x %/% 2
  }
  return(binary_string)
}
```

```
# 运行下面代码，验证结果
dec2bin(14) # 1110
```

```
[1] "1110"
```

```
dec2bin(123) # 1111011
```

```
[1] "1111011"
```

```
dec2bin(2203) # 100010011011
```

```
[1] "100010011011"
```

题目九:

```
# 在下面空白处定义函数mysum
mysum <- function(x){
  if(x==0){
    return(0)
  }else{
    return(x +mysum(x-1))
  }
}

# 测试
mysum(100)
```

[1] 5050

题目十:

```
# 在下面空白处定义函数myrecur
myrecur <- function(x, n) {
  x_str <- as.character(x)
  if (n == 1) {
    return(as.numeric(substr(x_str, nchar(x_str), nchar(x_str))))
  } else {
    return(myrecur(x %/% 10, n - 1))
  }
}

# 测试
myrecur(3627854,3) #8
```

[1] 8

```
myrecur(3627854,6) #6
```

[1] 6

题目十一:

```
# 在下面空白处，定义函数mygcd
mygcd <- function(a,b){
  r <- a%%b
  a <- b
  b <- r
  if (r == 0){
    return(a)
  }else{
    return(mygcd(a,b))
  }
}

# 测试
mygcd(24,32)
```

[1] 8

```
mygcd(32,24)
```

[1] 8

